Name : Atharva Ambilwade
PRN : 25070122055
Class : CSE
Batch : 2025-29

Assignment : 27

Subject: Programming with Java

ASSIGNMENT No : 27

TITLE : Develop a Java application demonstrating socket programming for communication between a client and a server.

Problem Statement : Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives : To establish communication between client and server using Java sockets.

Theory :

Socket programming enables communication between computers over a network using TCP/IP protocols. Java provides the Socket class for client-side communication and the ServerSocket class for server-side communication. A server listens for client requests on a specific port, while clients establish connections using the server's IP address and port number. Once connected, data is exchanged using input and output streams. Socket programming supports reliable communication between distributed systems and is widely used in chat applications, online gaming, file transfer systems, remote monitoring, and client-server architectures.

Code :

Server :

```java
import java.io.*;
import java.net.*;

public class Server {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5001);
            System.out.println("Server started. Waiting for client...");

            Socket s = ss.accept();
            System.out.println("Client connected.");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream())
            );

            PrintWriter out = new PrintWriter(
                s.getOutputStream(), true
            );

            System.out.println("Client: " + in.readLine());

            out.println("Hello Client, message received!");

            s.close();
            ss.close();
        }
        catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output :

Client :

```java
import java.io.*;
import java.net.*;

public class Client {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5001);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );

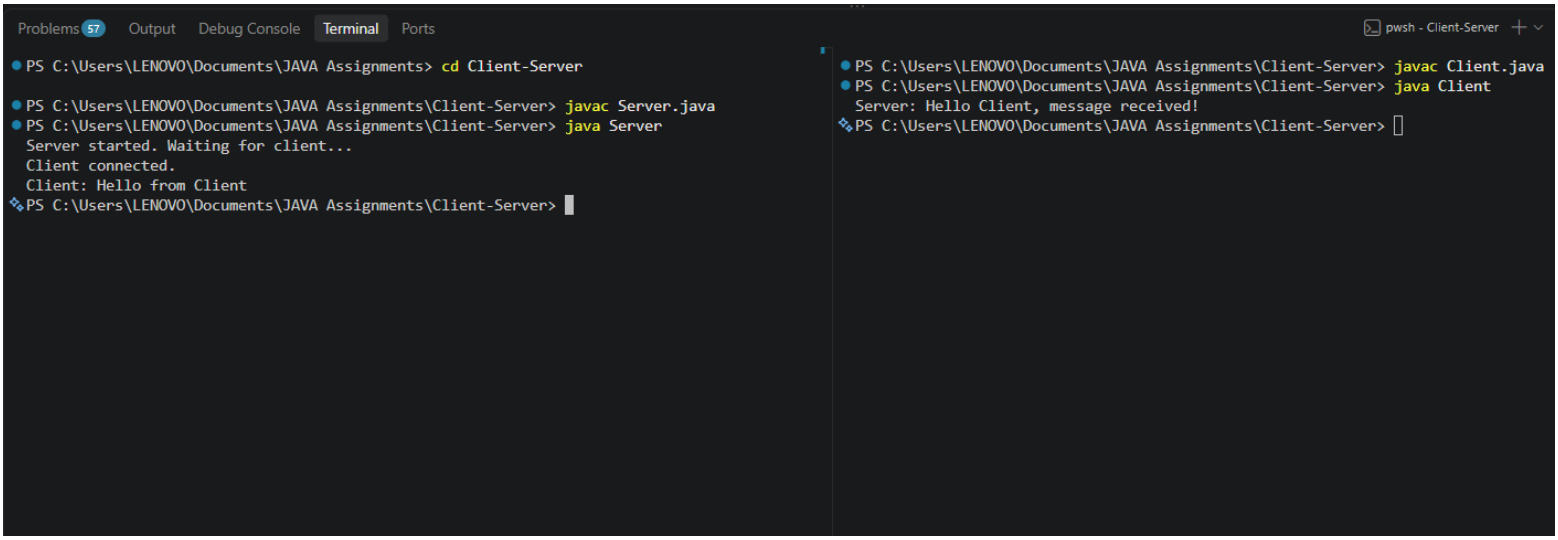
            PrintWriter out = new PrintWriter(
                socket.getOutputStream(), true
            );

            out.println("Hello from Client");

            System.out.println("Server: " + in.readLine());

            socket.close();
        }
        catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output :



Conclusion : The client-server program demonstrates communication between two Java applications using socket programming. The server uses ServerSocket to listen for client connections, while the client uses Socket to connect to the server. The BufferedReader and PrintWriter classes enable message exchange between them, demonstrating basic network communication in Java.

Exercise

1.Create a simple client-server chat application using sockets.
Ans) Server :

```java
import java.io.*;
import java.net.*;

public class Server {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5001);
        System.out.println("Server waiting for client...");

        Socket s = ss.accept();
        System.out.println("Client connected!");

        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));

        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);

        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in));

        String msg;

        while (true) {
            msg = in.readLine();
            System.out.println("Client: " + msg);

            if (msg.equalsIgnoreCase("bye"))
                break;

            System.out.print("Server: ");
            msg = keyboard.readLine();
            out.println(msg);

            if (msg.equalsIgnoreCase("bye"))
                break;
        }

        s.close();
        ss.close();
    }
}
```

Client :

```java
import java.io.*;
import java.net.*;

public class Client {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5001);
        System.out.println("Connected to server!");

        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));

        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);

        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in));

        String msg;

        while (true) {
            System.out.print("Client: ");
            msg = keyboard.readLine();
            out.println(msg);

            if (msg.equalsIgnoreCase("bye"))
                break;

            msg = in.readLine();
            System.out.println("Server: " + msg);

            if (msg.equalsIgnoreCase("bye"))
                break;
        }

        s.close();
    }
}
```

Output :



2. Create a Java program using socket programming where the client sends a message to the server, and the server returns the same message back (echo system).
Ans) Server :

Client-Server > Server.java > Java > Server

```java
import java.io.*;
import java.net.*;

public class Server {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5001);
        System.out.println("Server waiting...");

        Socket s = ss.accept();

        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));

        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);

        String msg = in.readLine();
        System.out.println("Client: " + msg);

        out.println(msg);

        s.close();
        ss.close();
    }
}
```

Client :

Client-Server > Client.java > Java > Client

```java
import java.io.*;
import java.net.*;

public class Client {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5001);

        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));

        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);

        out.println("Hello Server!");

        System.out.println("Server: " + in.readLine());

        s.close();
    }
}
```

Output :

```
Problems 53  Output  Debug Console  Terminal  Ports                                   pwsh - Client-Server

PS C:\Users\LENOVO\Documents\JAVA Assignments> cd .\Client-Server\
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> javac Server.java
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java Server
Server waiting...
Client: Hello Server!
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server>

PS C:\Users\LENOVO\Documents\JAVA Assignments> cd .\Client-Server\
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> javac Client.java
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server> java Client
Server: Hello Server!
PS C:\Users\LENOVO\Documents\JAVA Assignments\Client-Server>
```